

Continuous, rapid planning in humans

Jay Hennig

November 2025

Introduction

Decision-making, as studied in the lab, often focuses on how animals make discrete, one-off decisions that do not require planning. For example, a participant may be shown a single image and asked whether it is a cat or a dog. By contrast, humans and other animals must make decisions continuously, in an environment that is constantly changing. For example, while driving we often choose between different viable routes, and we may suddenly decide to take a new route if the traffic on our current route worsens. These decisions often have long-term consequences, such that we must consider the future outcomes of each decision. One approach to making such sequential decisions is called *planning*.

While a number of studies have begun to probe how humans make sequential decisions, these tasks have a few notable shortcomings. First, they often first require participants to memorize and/or learn the task's complex structure, such as the transitions or reward rates associated with different states. As a result, it is unclear to what extent suboptimal decisions in these tasks are due to incomplete learning of the task rather than poor planning. Second, these tasks are low-throughput, consisting of roughly one (binary) decision per second. This greatly limits the amount of the task's state space we can sample during a given session, and in turn our ability to use that data to rule out competing hypotheses about planning.

Falldown: a task for probing mechanisms of rapid, continuous planning in humans

In Falldown, the player continuously controls the horizontal movements of a ball (e.g., using the left and right arrow keys) to move the ball so that it can fall through holes in the floor. The player's goal is to reach the ground floor as quickly as possible by finding the most efficient path through the successive floors. There is also an immediate time pressure because the camera is continuously drifting down, and if the ball touches the top of the screen, the game is over.

At each decision point (e.g., dashed circle in Fig 1), players must make a decision (e.g., move left or right). Players may make their decision greedily, choosing the direction that is closest to the next hole (e.g., R_2), or they may make their decision by *planning*, e.g., by minimizing the distance traveled to get to future holes (e.g., L_2).

Benefits of this paradigm:

1. High-throughput (decisions happen rapidly)
2. Decisions are continuous (more naturalistic)
3. Can study learning by fixing the exact sequence of floors and letting patients play the task over and over
4. By studying the task online in hundreds of participants, we may be able to phenotype patients' executed sequences (e.g., infer their objective function)
5. Could study approach/avoidance by including rewards/punishments (as circles placed at different levels) that players could choose to visit/avoid along the way
6. Could study neural basis of planning by seeing whether future actions are encoded differently than concurrent actions

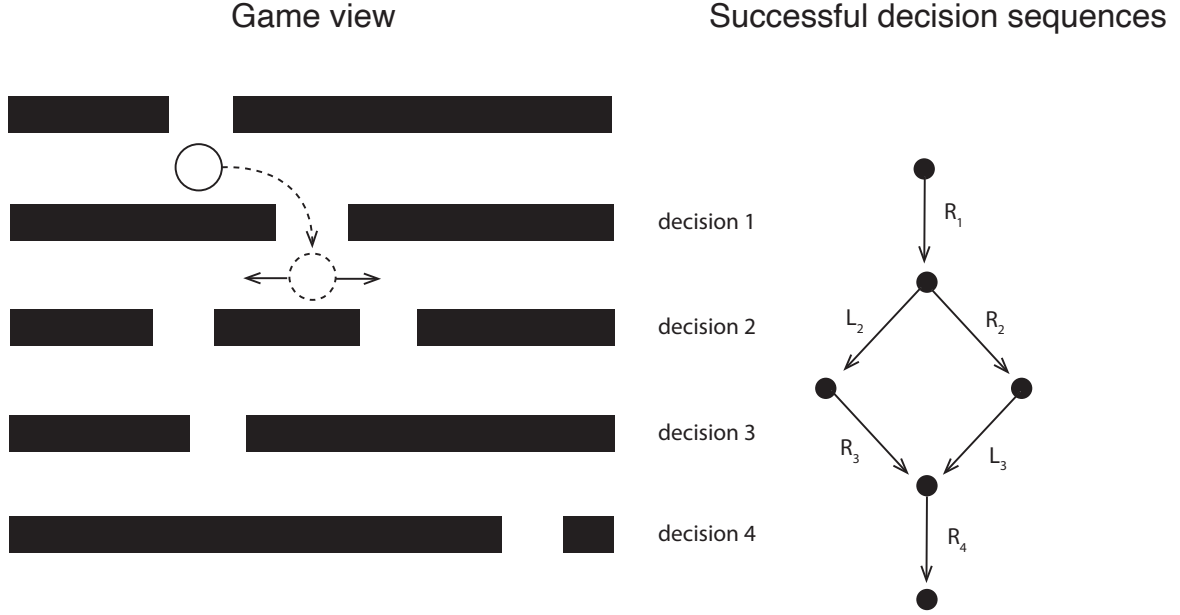


Fig 1. Left panel. The game as seen by the player (except for the arrows and dashed circle), made up of floors (black rectangles) and the player's ball (white circle). Curved dashed arrow denotes the ball's trajectory following a right movement, while the two solid arrows denote the player's choice at the next decision point. **Right panel.** Successful decision sequences that would allow the ball to make it through the floors shown in the left panel as quickly as possible. While different successful decision sequences are possible, these sequences may have different associated costs (e.g., total distance traveled).

Task design

The state space of Falldown, consisting of the sequence of floors and the location of each hole, is exponential in the number of total floors. As such, it is infeasible to fully sample every possible sequence of floors. The goal is to design the number and location of the holes at each level (i.e., floor), as well as how many levels ahead are currently visible, in order to efficiently test different hypotheses about how people make their decisions. For example, given a “budget” of $T = 1000$ levels (e.g., due to time constraints on the length of the session), we might ask what is the optimal series of levels that will help us best tease apart our competing hypotheses about how people are making their decisions. For example, we might want to know whether participants make their decisions greedily, or whether they make decisions based on the floors two, three, or even more steps ahead. Some potential hypotheses for how people make their decisions:

1. minimize total path length to get through the next N levels (where N may differ per player, or even over time for the same player); note that $N = 1$ is the strategy of “move toward the closest hole”
2. minimize changes in direction (aka there is some cost for having to alternate actions)
3. state-independent, chunk-based strategies, such as i) alternate directions on each level, or ii) move in same direction until you get too close to a wall, then move opposite

Let's suppose each level is made up of K equal-width blocks/sprits. Then we can parameterize the holes on level i as a binary vector, $x_i \in \{0, 1\}^K$. Then we can define $h_i = \sum_k x_i(k)$ as the number of holes in level i . For example, in Fig 1 we have the level sequence $h_0 = 1, h_1 = 1, h_2 = 2, h_3 = 1, h_4 = 1$.

To generate the full level sequence, we might create sequences of levels in chunks, where each chunk serves to test two different hypotheses. For example, suppose we have $h_i = 1, h_{i+1} = 2, h_{i+3} = 1$. Then as we did in Fig 1, we can choose the positions of these holes to directly distinguish a 1-step strategy from a 2-step strategy (as depicted in Fig 1). Alternatively, this same hole-order could test the inferred cost of direction changes by arranging the holes on level $i + 1$ so that a path without direction changes (e.g.,

R_2 would be the end of a right-right sequence) has a slightly longer length than a path with direction changes (e.g., L_2 would be the end of a right-left sequence).